

☕ Notes on Strings in Java

Strings in Java are a fundamental data type used to represent sequences of characters. They are widely used in programming for handling text, and understanding how to manipulate strings is essential for any Java developer. This document provides an overview of strings in Java, including their characteristics, common operations, and examples.

Understanding Java Strings

Characteristics

Properties and attributes of strings

Common Operations

Basic actions performed on strings

Examples

Practical applications of string usage



Made with  Napkin

Overview of Strings

- **Definition:** A string is a sequence of characters, which can include letters, digits, symbols, and whitespace.
- **Immutable:** Strings in Java are immutable, meaning once a string object is created, it cannot be changed. Any modification results in the creation of a new string object.
- **String Class:** The String class in Java provides various methods to manipulate strings.

Creating Strings

Strings can be created in two ways:

1. **Using String Literal:**

```
String str1 = "Hello, World!";
```

2. Using the new **Keyword**:

```
String str2 = new String("Hello, World!");
```

Common String Operations

1. Length of a String

To find the length of a string, use the `length()` method.

```
String str = "Hello";  
int length = str.length(); // length = 5
```

2. Accessing Characters

You can access individual characters in a string using the `charAt()` method.

```
char ch = str.charAt(1); // ch = 'e'
```

3. String Concatenation

Strings can be concatenated using the `+` operator or the `concat()` method.

```
String str1 = "Hello";  
String str2 = "World";  
String result = str1 + " " + str2; // result = "Hello World"
```

4. Substring

To extract a portion of a string, use the `substring()` method.

```
String str = "Hello, World!";  
String sub = str.substring(7, 12); // sub = "World"
```

5. String Comparison

Strings can be compared using the `equals()` method or `compareTo()` method.

```
String str1 = "Hello";
String str2 = "hello";
boolean isEqual = str1.equals(str2); // isEqual = false
int comparison = str1.compareTo(str2); // comparison < 0 (because 'H' < 'h')
```

6. String Conversion

You can convert other data types to strings using the `String.valueOf()` method.

```
int num = 100;
String strNum = String.valueOf(num); // strNum = "100"
```

7. String Searching

To find the position of a character or substring, use the `indexOf()` method.

```
String str = "Hello, World!";
int index = str.indexOf("World"); // index = 7
```

8. String Replacement

To replace characters or substrings, use the `replace()` method.

```
String str = "Hello, World!";
String newStr = str.replace("World", "Java"); // newStr = "Hello, Java!"
```

9. String Trimming

To remove leading and trailing whitespace, use the `trim()` method.

```
String str = "  Hello, World!  ";
String trimmedStr = str.trim(); // trimmedStr = "Hello, World!"
```

Conclusion

Strings are a crucial part of Java programming, and understanding their properties and methods is essential for effective text manipulation. The immutability of strings, along with the various operations provided by the String class, allows developers to handle text efficiently. By mastering string operations, you can enhance your Java programming skills and create more robust applications.